CyberSentinel

Enterprise Engineering & Security Audit

Independent Architecture Review Board • Fortune-500 Readiness Assessment

VERDICT: NOT PRODUCTION-READY — PROMISING ASM CORE ON A PRE-MVP FOUNDATION

Scope: full repository — FastAPI API, Go workers, React SPA, reporting, infra, docs

Method: line-level inspection of the actual codebase (every finding cites file:line)

Reviewers: Architecture, Backend, Frontend, Security, ASM/VA, SRE, DevSecOps, Cloud, DBA, QA, CTO

Repository: github.com/pankajneema/cyberSentinel • Date: June 2026

1 Executive Summary

CyberSentinel is an ambitious unified cybersecurity platform whose **Attack Surface Management (ASM) module is a genuine, database-backed product** with a real distributed scanning engine (15+ recon tools orchestrated through Go workers, RabbitMQ and PostgreSQL). That core is the company's strongest asset and is meaningfully ahead of where most early-stage teams are. However, the surrounding platform is **pre-MVP**, and the gap between the product's enterprise marketing ('enterprise-grade,' 'Kubernetes,' 'Terraform,' six modules) and the inspected reality is severe.

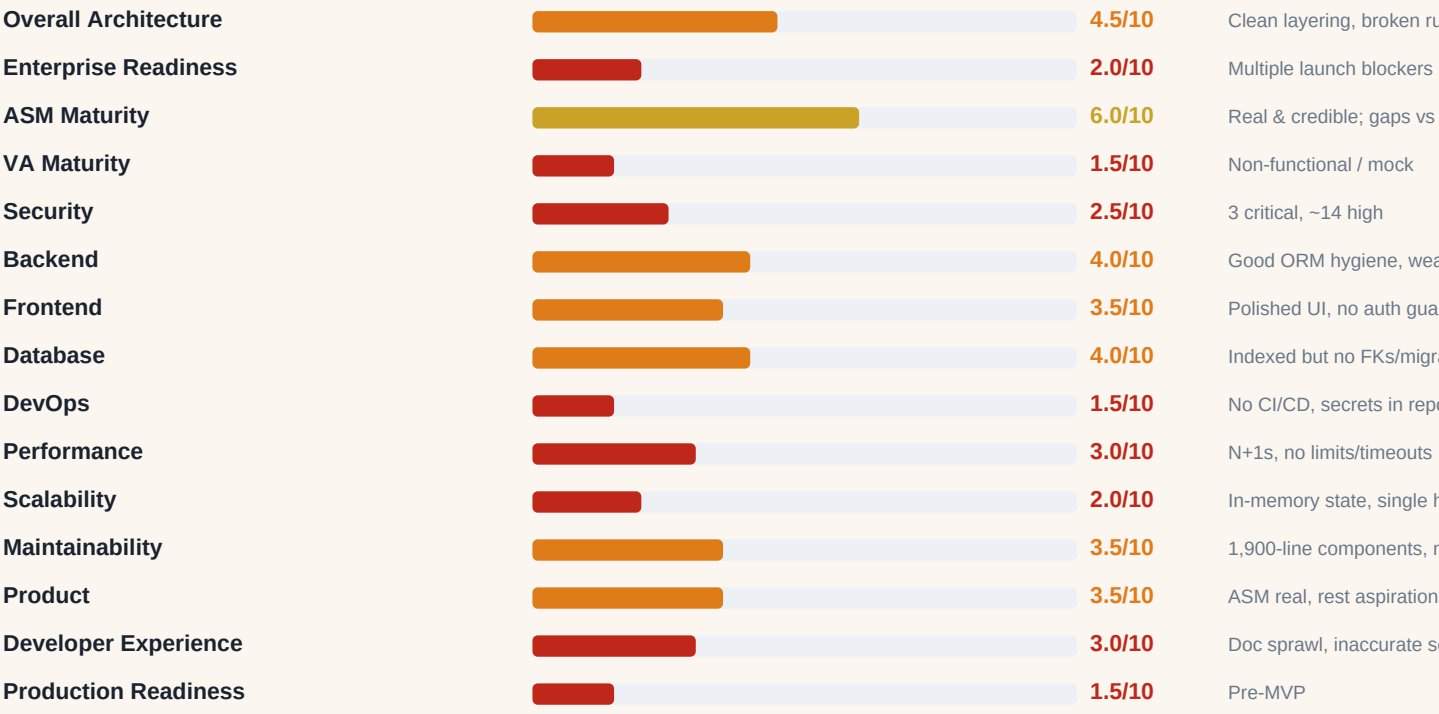
This review was conducted as a line-by-line inspection of the actual repository. Every finding cites a specific file and line. The board identified **three independently launch-blocking security defects**: (1) JWT signing keys default to hardcoded, committed placeholder values, allowing any party to forge admin tokens for any tenant; (2) a cross-tenant Broken Object-Level Authorization (IDOR) flaw in user management permits reading, modifying and deleting users across organizations; and (3) unvalidated scan targets enable SSRF, letting the platform be weaponized against internal/cloud-metadata endpoints.

Beyond security, the platform has **zero automated tests of any kind**, **no CI/CD**, **no real database migration tooling** (it uses `create_all()`), secrets committed in plaintext with no `.gitignore`, container images that compile from source or run dev servers in production, and Kubernetes/Terraform manifests that are non-functional stubs referencing services that do not exist. The **Vulnerability Assessment (VS) module does not actually scan** — it is a polished UI over hardcoded findings and an in-memory backend stub (`# TODO: Queue scan job`). Four of the six advertised modules (BAS, Threat Intel, Incident Response, Compliance) exist only as marketing copy.

Bottom line: The ASM engine demonstrates real engineering capability and is worth investing in. But the product as a whole cannot today be sold to, or survive the security audit of, a Fortune-500 customer. With focused remediation, the security blockers and DevOps foundation are fixable in roughly a quarter; achieving true multi-module enterprise parity is a 9–12 month program.

Enterprise Scorecard

Weighted assessment across the dimensions requested. Scores are evidence-based and deliberately harsh — this is a Fortune-500 readiness bar, not a hackathon rubric.



Overall Architecture	4.5/10	Clean layering, broken ru
Enterprise Readiness	2.0/10	Multiple launch blockers
ASM Maturity	6.0/10	Real & credible; gaps vs
VA Maturity	1.5/10	Non-functional / mock
Security	2.5/10	3 critical, ~14 high
Backend	4.0/10	Good ORM hygiene, wea
Frontend	3.5/10	Polished UI, no auth gua
Database	4.0/10	Indexed but no FKs/migr
DevOps	1.5/10	No CI/CD, secrets in repo
Performance	3.0/10	N+1s, no limits/timeouts
Scalability	2.0/10	In-memory state, single h
Maintainability	3.5/10	1,900-line components, r
Product	3.5/10	ASM real, rest aspiration
Developer Experience	3.0/10	Doc sprawl, inaccurate s
Production Readiness	1.5/10	Pre-MVP

Composite enterprise-readiness index: ~3.0 / 10 — a strong discovery engine embedded in a prototype platform.

2 Strengths (Verified in Code)

The board is brutally honest about defects below, so it is equally important to credit what is genuinely well done — and several things are:

- **ASM is a real product, not a mock.** 25 tenant-scoped endpoints (`routes/asm.py`) backed by SQLAlchemy models with correct object-level authorization (ownership verified via company user-id sets before returning child records).
- **Distributed scanning engine works.** Go workers orchestrate 15+ industry tools (subfinder, amass, naabu, nmap, nuclei, httpx, katana, sslscan, cloudenum, etc.) through a clean consumer / control-plane / executor layering.
- **No SQL injection and (almost) no command injection.** The API uses parameterized SQLAlchemy throughout; Go tool wrappers use argument-slice `exec` rather than shell strings (one `bash -lc` exception, currently mitigated by IP validation).
- **Sensible data model fundamentals.** Composite unique constraints and indexes on hot columns; connection pooling with `pool_pre_ping`; tenacity-based retry in the repository layer.
- **Structured logging in the Go workers** (zap, JSON production config) — the one bright spot in observability.
- **Polished, modern frontend shell.** React + Vite + shadcn/Radix gives baseline accessibility; ASM analyst views (discovery, inventory, runs, geo-map, attack-surface graph) are real and consume live APIs.
- **Honest internal positioning.** Project documentation acknowledges early-stage status rather than fabricating social proof — a healthy engineering culture signal.

3 Critical Risks (Launch Blockers)

Any one of the following is independently sufficient to fail an enterprise security review. All are confirmed in code.

CRITICAL	**C-1. Forgeable JWTs — hardcoded signing secrets**	`config/settings.py:40-47; .env:28-31`
Issue	`JWT_SECRET/SECRET_KEY` fall back to literal strings ('your-jwt-secret-change-in-production') and the committed .env ships trivial values ('jwtsecretkey'). Tokens are HS256-signed, so anyone who knows/guesses the key can forge a valid token for any `user_id` in any tenant.	
Impact	Complete authentication bypass and full cross-tenant account takeover. For a security product this is a catastrophic, reputation-ending breach class.	
Fix	Remove fallbacks; fail-fast at startup if secret is unset or <32 bytes; rotate leaked keys; consider RS256 for multi-service verification.	
Effort / Std	0.5 day | *OWASP A02/A07; CWE-798/CWE-321; NIST SP 800-57*	

CRITICAL	**C-2. Cross-tenant IDOR in user management**	`routes/users.py:51-208`
Issue	`list_users` selects all users with no tenant filter; `get/update/delete_user` resolve targets by id with only a *global* admin check and never verify the target shares the caller's `company_id`. An admin of Company A can enumerate, re-role, and delete users of Company B.	
Impact	Cross-tenant data disclosure plus destructive cross-tenant writes and privilege escalation — a multi-tenant SaaS showstopper.	
Fix	Scope every query by caller's company_id; verify target company_id equality before any read/update/delete; remove the unbounded listing.	
Effort / Std	1 day | *OWASP A01 / API1:2023 (BOLA); CWE-639/CWE-284*	

CRITICAL	C-3. SSRF via unvalidated scan targets	schemas/asm_schema.py:13; routes/asm.py:194-245
Issue	`manual_targets` accepts arbitrary strings with no validation and is enqueued straight to the scanning workers. No block on localhost, 127.0.0.1, RFC-1918, link-local `169.254.169.254` (cloud metadata) or `file://`.	
Impact	A tenant can turn the platform's own scanners against internal infrastructure or cloud-metadata endpoints, or scan third-party assets they do not own (legal/abuse exposure for an unauthorized-scanning service).	
Fix	Validate/normalize targets; block private/link-local/metadata ranges; require domain ownership proof; enforce per-tenant scope before enqueueing.	
Effort / Std	2-3 days | *OWASP A10 (SSRF); CWE-918*	

CRITICAL	C-4. Committed secrets, no .gitignore	backend/api_service/.env, backend/workers/.env; (no .gitignore in repo)
Issue	Live-style DB, SMTP, RabbitMQ and JWT secrets sit in plaintext .env files; the reporting Dockerfile (`COPY backend /app/backend`) bakes them into image layers. A real Gmail app password is hardcoded as a default in `notificationservice/email.py:13`.	
Impact	One `git add .` permanently leaks credentials; secrets already exposed to anyone with repo/image access. Fails every secret-scanning and SOC2 gate.	
Fix	Rotate all secrets now; add .gitignore + .env.example; adopt a secrets manager (Vault/AWS SM); add .dockerignore; scrub history.	
Effort / Std	1 day | *CWE-540/CWE-798; OWASP A05; NIST IA-5*	

CRITICAL	C-5. Jobs silently lost on worker crash; no tests; no recovery	orchestration/job_manager.go:183; consumer/asm/job.go:101; executor/runner/task.go
Issue	The AMQP message is ACKed after a synchronous HTTP handoff, then the real scan runs in a detached goroutine (`go executeJob()`) with no link to the delivery, no panic recovery, no resume checkpoint and no timeout. A crash mid-scan loses the paid job and strands it at RUNNING forever. The entire worker subsystem has zero tests.	
Impact	Silent data loss of customer scans, stuck jobs, and no change-management confidence — unacceptable for enterprise SLAs.	
Fix	ACK only after terminal success; make execution synchronous or use manual-ack + heartbeat; add per-step checkpoints, panic recovery, timeouts and a stale-job reaper; build a test harness.	
Effort / Std	High (1-2 wks) | *Reliability / at-least-once processing; SOC2 CC8.1*	

4 Security Risks (OWASP / CWE Mapped)

Beyond the critical blockers, the following high-severity security gaps were confirmed:

HIGH	S-1. No authentication guard on the SPA	`frontend/src/App.tsx:48-60; layouts/AppLayout.tsx`
Issue	No `ProtectedRoute`/`RequireAuth` exists. Every `/app/*` route mounts without a token check; protection is 'by accident' via API 401s. RBAC is client-trusted (`canWrite` prop defaults to true).	
Impact	Internal UI, labels and customer-data-shaped views leak to unauthenticated users; client-side role gates are trivially bypassed.	
Fix	Add a RequireAuth wrapper that verifies the token server-side and redirects on failure; drive RBAC from a verified server role.	
Effort / Std	Medium | *OWASP A01; SOC2 access control*	

HIGH	S-2. JWT stored in localStorage; broken refresh/reset/logout	`Login.tsx:28; routes/auth.py:161-247`
Issue	Token in `localStorage` (XSS-stealable). Server-side `refresh_token` returns a literal fake token; `reset_password` reports success while doing nothing; logout is a no-op with no revocation/denylist.	
Impact	Token theft via any XSS; no working credential recovery or session renewal; removed members keep access until natural expiry.	
Fix	Move JWT to httpOnly cookie (credentials already half-wired); implement signed single-use reset/refresh tokens with rotation + Redis denylist.	
Effort / Std	Medium-High | *OWASP A07; CWE-522/CWE-613/CWE-640*	

HIGH	S-3. No rate limiting anywhere	`routes/auth.py:113-155 (whole app)`
Issue	Login and all endpoints have no throttling despite Redis being available. No account lockout/backoff.	
Impact	Unlimited credential stuffing, email enumeration, and resource-exhaustion abuse.	
Fix	Add Redis-backed limiter (slowapi/fastapi-limiter) on auth + write paths; add lockout/backoff.	
Effort / Std	1 day | *OWASP API4:2023; CWE-307*	

HIGH	S-4. CORS misconfigured with credentials	`config/settings.py:69-71; main.py:29-35`
Issue	`allow_origins` receives a bare string (iterated per-character) with `allow_credentials=True` and wildcard methods/headers; the common 'fix' to `['*']` is the worst case.	
Impact	Either broken browser auth or cross-origin credential exposure.	
Fix	Parse a comma-separated allow-list; never combine wildcard origin with credentials.	
Effort / Std	0.25 day | *OWASP A05; CWE-942*	

HIGH	S-5. Internal errors / stack traces leaked to clients	`routes/auth.py:102-107; routes/users.py:76-80`
Issue	Endpoints return `str(e)` (DB constraint names, driver messages) in responses; no global exception handler; no security headers (HSTS/CSP/X-Frame-Options).	
Impact	Information disclosure that aids further attacks; inconsistent error contract.	
Fix	Return generic messages with a correlation id; add a global exception handler and a security-headers middleware.	
Effort / Std	0.5 day | *OWASP A05/API8; CWE-209/CWE-693*	

HIGH	**S-6. Supply chain — security tools installed @latest; root containers**	`workers/install-tools.sh; all four Dockerfiles`
Issue	Every scanner is `go install ...@latest` or `curl | sh`, unpinned, no checksums/SBOM. All images run as root; the worker image runs `go run` (compiles at start); frontend ships `vite preview` as prod.	
Impact	Non-reproducible builds; a compromised upstream release runs against customer networks with broad privileges.	
Fix	Pin every tool to a version + verify checksums; multi-stage non-root distroless images; generate an SBOM; scan images in CI.	
Effort / Std	Medium | *SLSA; CIS Docker 4.x; NIST 800-190*	

5 Architecture, Performance & Product Risks

Architecture Risks

- **Non-scalable in-memory job registry** (`job_manager.go:19`, `map[string]*Job` behind a mutex). State is per-process and lost on restart; multiple control-plane replicas cannot share it — horizontal scaling is impossible as built. A `// TODO load balance and sharding` acknowledges this.
- **DB I/O held under a global write lock** (`RegisterJob`) serializes all job registrations behind one mutex during network I/O.
- **Tenant identity rebuilt per request** rather than stored — ASM/asset rows key on user_id, not company_id, and every endpoint re-derives the company user set (N+1, fragile, easy to forget — as users.py proves).
- **Single-host runtime** — the real deploy path `nohup`s uvicorn + `go run` + a consumer on one machine; no supervisor, no restart-on-crash, no LB.
- **Schema management via create_all()** — no Alembic; any post-launch column change needs manual SQL and risks data loss.

Performance Risks

- **Reporting N+1 queries** (`domain.py` per-port / per-service SELECTs; `ip.py` per-IP upserts; eight full-table snapshot loads) — will not scale to large attack surfaces.
- **No timeouts or cancellation** — a defined `TaskTimeoutSec` is never used; nmap deep scans run `-p-` (all 65,535 ports) unbounded; a hung tool blocks a worker indefinitely.
- **No resource limits** in compose or K8s (no CPU/memory bounds) — DoS / noisy-neighbor risk.
- **No QoS/prefetch and no dead-letter queue** on RabbitMQ — a poison message requeues forever.
- **Per-process connection pools** hardcoded with no external pooler (PgBouncer) — connection storms at scale.

Product Risks

- **VS module is non-functional** — `create_scan` writes to a Python dict with `# TODO: Queue scan job`; the dashboard fabricates a synthetic `CVE-2024-0001 / CVSS 9.8`; findings are hardcoded in the UI. It cannot be demoed as a Nessus/Qualys alternative without misrepresentation.
- **Four of six advertised modules (BAS, TI, IR, Compliance) do not exist** in code — only landing-page copy.
- **'Promote to Asset' lifecycle** is referenced in settings flags but not implemented end-to-end in the UI.
- **Exposure scoring is magic numbers** (`len(ports)*3 + 12/sensitive-port`), not a defensible CVSS/EPSS-based model — indefensible under enterprise audit.
- **Fake 'live scan' widget** driven by `mockActiveScans` in the global header.

6 Missing Enterprise Features

Cross-cutting enterprise capabilities expected by a Fortune-500 buyer that are currently absent:

- SSO / SAML / OIDC, SCIM provisioning, MFA (only local email+password JWT exists; no OAuth/Supabase despite claims).
- Real RBAC/ABAC enforcement server-side, organization-level isolation column, and full audit trail / SIEM export.
- Tamper-evident audit logging, data-retention & residency controls, encryption-at-rest, TLS enforcement (`sslmode=disable` today).
- Soft-delete + recovery (all deletes are hard, cascade can destroy a user's entire asset history).
- Notifications/alerting pipeline, scheduling at scale, multi-tenant quotas/billing enforcement.

Missing ASM Features (vs Defender EASM / Xpanse / Censys / ProjectDiscovery)

Capability	Status	Note
Screenshots / visual recon	Absent	No gowitness/screenshot capture — a baseline EASM feature.
Technology fingerprinting	Absent	No Wappalyzer-style tech/version mapping.
Ownership verification	Absent	No DNS-TXT/ownership proof before scanning external assets.
Certificate expiry alerting	Partial	SSL inventory stored, but no expiry/rotation alerting.
Change/delta operationalization	Partial	Changes persisted; no diff visualization or alert pipeline.
Continuous monitoring scheduler	Partial	Schedule fields exist; running scheduler not evidenced.
Asset relationship graph API	Partial	Graph derived client-side; no dedicated edges API.

Missing VA Features (vs Qualys / Nessus / Rapid7 / OpenVAS)

Capability	Status	Note
Scan execution pipeline	Stub	create_scan only writes a dict; # TODO queue job.
Plugin / check architecture	Absent	No NVT/template/plugin concept.
CVE mapping & CVSS scoring	Fake	Single hardcoded CVE-2024-0001; literal CVSS 9.8.
Finding lifecycle / persistence	Fake	Statuses exist only in a mock UI array; lost on restart.
False positives / suppression / exceptions	Absent	No FP/suppression/exception handling.
Evidence / PoC capture	Absent	None.
Risk prioritization (EPSS/criticality)	Absent	No exploit-maturity or asset-criticality model.

Missing Reporting Features

Capability	Status	Note
Generated report documents	Absent	Reporting service writes DB rows only; no PDF/exec/customer reports.
Executive / developer / customer views	Absent	No templated report tiers.
Historical / scheduled reports	Stub	UI lists are hardcoded; no backend generation/scheduling.
Idempotent ingestion	Partial	Append-only change rows + dedup keys missing tenant/discovery scope.

7 Technical Debt & Refactoring

Technical Debt (highest-interest first)

- **Zero tests** across FastAPI, Go workers and React — and docs falsely claim `npm test`/`pytest` work. This is the debt that makes every other fix risky.
- **God components** — `DiscoveryManager.tsx` (1,901 lines), `AssetInventory.tsx` (1,108) mix fetch, state, dialogs and rendering.
- **TypeScript strictness disabled** (`strict:false, strictNullChecks:false`) — negates TS safety value for a security product.
- **Copy-paste route logic**, debug `print()` in prod paths, deprecated `@app.on_event` hooks, stub endpoints reporting success.
- **Mock data baked into production components** (VS dashboard/findings, asm Vulnerabilities, reports, live-scan widget) — must be deleted and wired to APIs.
- **Doc sprawl** — six overlapping run/setup guides (one empty), referencing ClickHouse, an `auth-service` and paths that do not exist.

Refactoring Recommendations

- Introduce a **company_id tenant column** on assets/ASM/finding rows and filter directly; add real foreign keys with `ondelete` across ASM tables.
- Make **PostgreSQL/Redis the source of truth** for job state; remove the in-memory registry; ACK after success.
- Extract a **shared service layer** for current-user/tenant resolution via FastAPI dependency injection (one place to enforce authz).
- Adopt **react-query** (already installed) for all data fetching; decompose 1,000+ line components; turn on TS strict incrementally.
- Replace the **exposure scoring** heuristic with a documented, configurable CVSS/EPSS-weighted model.

8 Prioritized Roadmap

Quick Wins — 1–2 weeks (do first)

- Rotate & remove all secrets; add `.gitignore` + `.env.example` + `.dockerignore`; fail-fast on missing JWT secret (fixes C-1, C-4).
- Patch the users.py cross-tenant IDOR; add tenant scoping to every query (C-2).
- Validate/allow-list scan targets, block private/metadata ranges (C-3).
- Fix CORS to a parsed allow-list; add security-headers middleware; stop leaking `str(e)` (S-4, S-5).
- Add Redis-backed rate limiting on auth (S-3); add a RequireAuth route guard (S-1).
- Replace the `bash -lc` nmap call with arg-slice exec; add panic recovery + the existing TaskTimeoutSec.

Medium Improvements — 1–2 months

- Stand up CI/CD: build + lint + SAST + dependency & container scan + secret scan; bootstrap pytest/vitest/go-test with smoke tests gating merges.
- Implement real auth: httpOnly-cookie sessions, working refresh/reset with rotation + denylist, optional OIDC/SAML SSO.
- Build a functioning VS scan pipeline (worker + queue + DB persistence); delete all mock data; real CVE/CVSS mapping.
- Adopt Alembic migrations; add company_id tenant column + FKs; soft-delete + audit tables.
- Rewrite Dockerfiles (multi-stage, non-root, distroless, compiled Go binary, nginx static frontend, healthchecks); pin tool versions + SBOM.
- Add baseline observability: Prometheus `/metrics`, Sentry, log shipping, deep health/readiness.

Long-Term — 6–12 months

- Move job state to a distributed store; enable true horizontal scaling, HPA, and real K8s manifests + functioning Terraform (EKS, encrypted RDS, remote state, multi-AZ, backups/DR).
- Close ASM gaps to leader parity: screenshots, tech fingerprinting, ownership verification, cert-expiry alerting, operationalized change detection, continuous monitoring.
- Mature VA into a real scanner (plugin architecture, suppression/exceptions, evidence, EPSS-based prioritization) and build the reporting tiers (executive/developer/customer, scheduled, exportable).
- Deliver the remaining advertised modules (BAS, Threat Intel, Incident Response, Compliance) or re-scope the marketing to match reality.
- Pursue SOC 2 Type II / ISO 27001: threat model, audit trail/SIEM, data-retention, runbooks, DR drills.

9 Comparison vs Enterprise Products

Honest positioning of CyberSentinel's ASM and VA against established platforms. 'Core engine' credits the real discovery pipeline; 'enterprise wrap' (auth, multi-tenancy, compliance, reporting, scale) is where the gap is widest.

Dimension	CyberSentinel	Defender EASM / Xpanse / Censys	Qualys / Nessus / Rapid7
Asset discovery breadth	Good (15+ tools)	Comprehensive, continuous, global	N/A (VM-focused)
Visual recon / screenshots	Absent	Standard	—
Tech fingerprinting	Absent	Standard	Standard
Vulnerability scanning	Non-functional (mock)	Add-on / partial	Mature, signature/plugin-based
CVE/CVSS/EPSS scoring	Hardcoded	Real	Mature
Continuous monitoring	Partial	Mature	Mature
Multi-tenancy & RBAC	Broken (IDOR)	Enterprise-grade	Enterprise-grade
Reporting (exec/customer)	Absent	Mature	Mature
Compliance (SOC2/ISO)	None	Certified	Certified
Scale (100k+ assets)	Not ready	Proven	Proven

CyberSentinel competes on **discovery engine quality** at an early stage but is a generation behind on the enterprise wrapper that buyers actually pay for. The fastest credible market entry is to **position narrowly as an ASM/EASM product**, fix the security blockers, and defer the multi-module 'platform' story until the foundation exists.

10 Final Verdict

Is CyberSentinel production-ready? No. It is pre-MVP for enterprise use.

Would the board approve it for enterprise customers today? No. Three independent critical defects (forgeable JWTs, cross-tenant IDOR, SSRF) each constitute a breach class on their own. Combined with zero tests, no CI/CD, secrets in the repo, and a non-functional VA module advertised as a scanner, the platform would fail the first external security assessment and expose the company to contractual and regulatory liability.

Blockers that must be fixed before any launch or pilot:

- Eliminate forgeable JWTs and rotate/remove all committed secrets (C-1, C-4).
- Fix the cross-tenant IDOR and enforce real server-side tenant isolation + RBAC (C-2, S-1).
- Validate scan targets to close SSRF and unauthorized-scanning abuse (C-3).
- Make the worker pipeline crash-safe (no lost jobs) and add at least smoke-level tests + CI gating (C-5).
- Stop shipping the VS module as a functional scanner until it actually scans, or clearly label it as preview.

As CTO, would I merge and deploy this today?

No — I would not deploy, and I would not merge to a release branch. The evidence is unambiguous: a security product cannot ship with default-forgeable authentication, cross-tenant data deletion, SSRF, plaintext secrets in git, and no test suite. Deploying would put both customers and the company at unacceptable risk.

However, I would **strongly back continued investment**. The ASM discovery engine is real, competently built, and the hardest part to get right — it is a credible foundation, not a façade. My decision would be: freeze enterprise/marketing claims to match reality, run the 1–2 week security sprint to clear the critical blockers, stand up CI and a test harness, and re-review. With disciplined execution, CyberSentinel can become a legitimate ASM product within a quarter and a defensible multi-module platform within a year. The talent and the core are here; the engineering rigor and security hardening are what must catch up.

Prepared by the Independent Architecture Review Board · All findings cite inspected source at github.com/pankajneema/cyberSentinel · Confidential.